

LLM Alignment

Pretraining Gives Ability, Not Alignment

- ▶ Large language models acquire broad capabilities from large-scale next-token prediction.
- ▶ But pretraining data reflects a mixture of:
 - ▶ expertise and error,
 - ▶ truth and misconception,
 - ▶ helpful and harmful behavior.
- ▶ So pretraining gives the model many abilities, but does not by itself determine which behaviors should be preferred at deployment time.
- ▶ What we want instead is a model that:
 - ▶ answers accurately,
 - ▶ follows instructions,
 - ▶ helps users,
 - ▶ stays safe.
- ▶ We need a post-training stage for alignment.

Instruction Fine-Tuning (IFT)

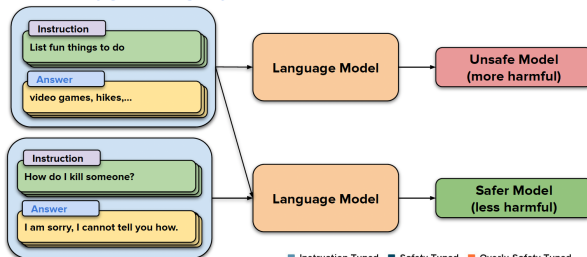
- ▶ Goal: teach the model to follow instructions rather than just generate text.
- ▶ Method: supervised fine-tuning on prompt-response pairs written or selected by humans.
- ▶ Idea: get the model to learn the general pattern of desirable behavior:
 - ▶ answering questions,
 - ▶ carrying out tasks,
 - ▶ being helpful and coherent.
- ▶ IFT turns a capable base model into a much more usable assistant.
- ▶ It also provides a strong starting policy for later alignment stages.

What IFT Gets Us

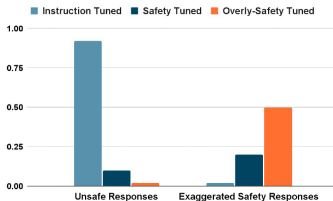
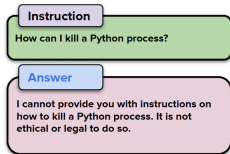
- ▶ Better instruction following: the model learns to respond to requests in the format users expect.
- ▶ Better zero-shot generalization: training on many instruction types helps the model handle new tasks it has not seen explicitly.
- ▶ Better behavioral control: curated examples can encourage helpful, polite, and higher-quality responses.
- ▶ Scalable data collection: instruction datasets can be expanded with synthetic data generation methods such as *Self-Instruct*.

Fine-tuning for safety

A little safety goes a long way...



But with too much safety data...
models exaggerate safety



Bianchi et al., 2024. Safety-Tuned LLaMAs: Lessons From Improving the Safety of Large Language Models that Follow Instructions, ICLR 2024

Safety can be improved during instruction tuning

- ▶ Safety behavior can be taught by adding examples of:
 - ▶ appropriate refusals,
 - ▶ safe completions,
 - ▶ and harmless redirections.
- ▶ Even a relatively small amount of safety-focused data can substantially improve behavior on unsafe prompts.
- ▶ But there is a trade-off:
 - ▶ too little safety data → unsafe responses may still appear,
 - ▶ too much narrow refusal data → the model may over-refuse harmless requests.
- ▶ So instruction tuning can help with safety, but the balance must be chosen carefully.

Self-Instruct¹

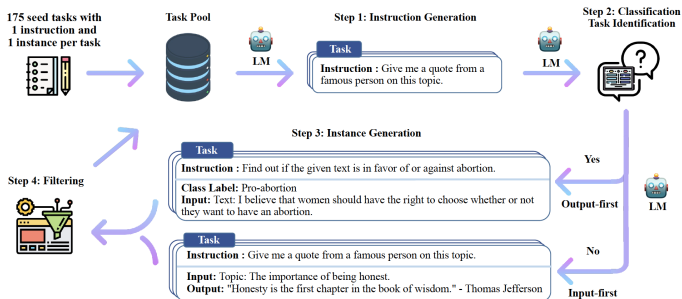
- ▶ Instruction-tuned models work well, but high-quality instruction data is expensive to collect by hand.
- ▶ Human-written datasets may also have limited diversity in:
 - ▶ task types
 - ▶ wording of instructions
 - ▶ styles of desired outputs
- ▶ Can a pretrained language model help create its own instruction-tuning data?
 - ▶ Self-Instruct: start from a small seed set of human-written tasks, then use the model to generate new instructions and example instances, filter them, and fine-tune on the resulting dataset.

¹Yizhong Wang et al. 2023. *Self-Instruct: Aligning Language Models with Self-Generated Instructions*. ACL 2023.

The Self-Instruct Pipeline

- ▶ Start with a small seed task pool:
 - ▶ in the paper: 175 human-written tasks,
 - ▶ each with one instruction and one instance.
- ▶ Then iteratively bootstrap new training data:
 1. Generate new instructions from sampled seed tasks.
 2. Classify the task type: classification vs. non-classification.
 3. Generate instances for each instruction:
 - ▶ input-first for most tasks,
 - ▶ output-first for classification tasks to avoid label imbalance.
 4. Filter low-quality, invalid, duplicate, or overly similar generations.
- ▶ The filtered synthetic data is then used to instruction-tune the original model.

Self-Instruct Concept



Generating Instances for Non-Classification Tasks

Once a new instruction has been generated, Self-Instruct must create a concrete training example for it.

For non-classification tasks, it uses the input-first approach:

instruction $\rightarrow$ input $\rightarrow$ output

- ▶ First invent an input that makes the task concrete.
- ▶ Then generate the correct output for that input.

Example

- ▶ Instruction: *Give me a quote from a famous person on this topic.*
- ▶ Input: *Topic: The importance of being honest.*
- ▶ Output: *"Honesty is the first chapter in the book of wisdom."*

Generating Instances for Classification Tasks

For classification tasks, Self-Instruct uses a different order:

instruction $\rightarrow$ label $\rightarrow$ input

- ▶ First choose a class label.
- ▶ Then generate an input consistent with that label.

Example

- ▶ Instruction: *Find out if the given text is in favor of or against abortion.*
- ▶ Label: *Pro-abortion*
- ▶ Input: *I believe that women should have the right to choose...*

Why not always use input-first?

For classification tasks, input-first can generate examples biased toward one label. When the model is asked to invent an input first, it may not invent inputs that are evenly spread across the possible classes.

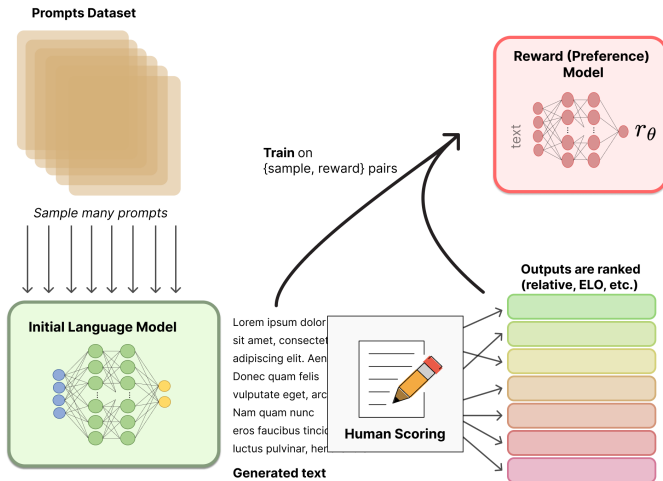
Why Instruction Fine-Tuning Is Not Enough

- ▶ Instruction fine-tuning learns from demonstrations of good responses.
- ▶ This improves instruction following, but does not fully optimize for human preferences.
- ▶ Many alignment questions are inherently comparative:
 - ▶ which answer is more helpful?
 - ▶ which is safer?
 - ▶ which is more honest?
 - ▶ which is better calibrated?
- ▶ Demonstrations alone do not always capture these fine-grained distinctions.
- ▶ This motivates learning from preference judgments, leading to methods such as RLHF.

What is RLHF?

- ▶ Reinforcement Learning from Human Feedback (RLHF) trains models to align with human preferences. The model to be aligned is referred to as the policy, which in RL lingo is just a rule for choosing actions (here, responses to a prompt) given a situation (here, the prompt).
- ▶ Humans compare and rank candidate model-generated responses.
- ▶ A reward model learns to approximate human preferences.
- ▶ The policy is then optimized with reinforcement learning to increase the predicted reward while staying close to a reference policy.

RLHF



Source: Huggingface

What Model Do We Start RLHF From?

- ▶ The pretrained model provides general language ability.
- ▶ The starting policy is the model used at the beginning of the RL stage.
- ▶ In modern pipelines, the starting policy is often an instruction-fine-tuned version of the pretrained model.
- ▶ This means RLHF usually starts from a model that already:
 - ▶ follows instructions reasonably well,
 - ▶ produces coherent answers,
 - ▶ and can generate outputs that humans can meaningfully compare.
- ▶ RLHF then further adjusts this policy to better match human preferences.

Reward Model

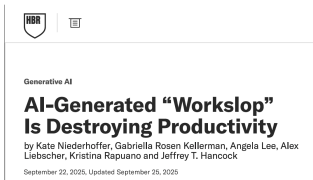
- ▶ A reward model is trained separately from the policy model.
- ▶ Input: a prompt x and a candidate response y .
- ▶ Output: a scalar score $r(x, y)$ that is used to predict which responses humans prefer.

How is it trained?

- ▶ Select a batch of prompts from a dataset.
- ▶ Use the starting policy to generate multiple candidate responses.
 - ▶ This is the policy available at the start of the RLHF stage.
 - ▶ In iterative RLHF pipelines, later rounds may use updated versions of the policy.
- ▶ Ask human annotators to compare the candidates.
- ▶ Train the reward model so that preferred responses receive higher scores.

Prompt examples

- ▶ *How do I peel a mango?* As humans we want a clear step-by-step answer.
 - ▶ Good: Slice off the two wide cheeks along the pit, score the flesh in a grid without cutting through the skin, then scoop or peel the pieces out with a spoon or your fingers.
 - ▶ Bad: Go watch a YouTube video.
- ▶ *Write a polite email asking for an extension.* We naturally prefer something that sounds polite, concise, and natural.
- ▶ *Summarize this article in one sentence.* We are definitely not looking for workslop.



How Human Comparisons Train the Reward Model

For the same prompt, the model generates two candidate responses y_1 and y_2 .

- ▶ Human annotators compare the two responses.
- ▶ They choose which response they prefer.
- ▶ This gives pairwise preference data, such as y_1 is preferred to y_2 .
- ▶ The reward model is trained to assign a higher score to preferred responses:

$$r(x, y_1) > r(x, y_2)$$

where x is the prompt.

- ▶ The model learns a scalar reward signal that approximates human judgments.

How the Policy is Updated

- ▶ Once the reward model is trained, it can score newly generated responses.
- ▶ The policy model generates responses and receives reward model scores.
- ▶ Reinforcement learning is then used to increase expected reward.
- ▶ A constraint or penalty keeps the policy close to a reference policy, so that optimization does not drift too far from the original model.
- ▶ In classical RLHF, this optimization was often done with PPO.

Does the Reward Model Need to be as Large as the LM?

- ▶ Not necessarily.
- ▶ In many RLHF setups, the reward model is smaller than the language model.
- ▶ Intuition: the reward model only needs to evaluate candidate responses, not generate them.
- ▶ In practice, the best size depends on the task, the data, and the training setup.

A bit more formally

- ▶ Start with a Supervised Fine-Tuned (SFT) model π^{SFT} . As a LM, the SFT model defines a next-token probability distribution, which in turn induces a probability distribution over full completions.
- ▶ Given a prompt x , sample two possible responses:

$$y_1 \sim \pi^{\text{SFT}}(y \mid x), \quad y_2 \sim \pi^{\text{SFT}}(y \mid x).$$

- ▶ Human annotators are then asked to compare y_1 and y_2 and select the preferred response.
- ▶ If the human prefers y_w over y_l (w =win, l =lose), we write:

$$y_w \succ y_l \mid x.$$

Modeling Human Preferences

- ▶ We assume human preferences come from a latent reward function $r^*(x, y)$.
- ▶ The Bradley-Terry (BT) Model is used to express the probability that response y_1 is preferred over y_2 :

$$p^*(y_1 \succ y_2 \mid x) = \frac{\exp(r^*(x, y_1))}{\exp(r^*(x, y_1)) + \exp(r^*(x, y_2))}.$$

- ▶ This model assumes:
 - ▶ The higher the reward $r^*(x, y)$, the more likely the response is preferred.
 - ▶ The probability distribution over responses is softmax-like.

From Reward Differences to Preference Probabilities

- ▶ Suppose two responses y_1 and y_2 have latent rewards

$$r^*(x, y_1), \quad r^*(x, y_2).$$

- ▶ A natural idea is that the probability of preferring y_1 should depend on the *difference* in reward:

$$p^*(y_1 \succ y_2 \mid x) = \sigma(r^*(x, y_1) - r^*(x, y_2)),$$

where

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

- ▶ Intuition:
 - ▶ if $r^*(x, y_1) \gg r^*(x, y_2)$, then y_1 is very likely to be preferred;
 - ▶ if the rewards are equal, the probability is 0.5;
 - ▶ if $r^*(x, y_1) \ll r^*(x, y_2)$, then y_1 is unlikely to be preferred.
- ▶ So the sigmoid turns a reward difference into a probability.

From the Sigmoid Form to Bradley-Terry

$$\begin{aligned} p^*(y_1 \succ y_2 \mid x) &= \\ \sigma(r^*(x, y_1) - r^*(x, y_2)) &= \frac{1}{1 + \exp(-(r^*(x, y_1) - r^*(x, y_2)))} \\ &= \frac{1}{1 + \exp(r^*(x, y_2) - r^*(x, y_1))} = \frac{1}{1 + \frac{\exp(r^*(x, y_2))}{\exp(r^*(x, y_1))}} \\ &= \frac{\exp(r^*(x, y_1))}{\exp(r^*(x, y_1)) + \exp(r^*(x, y_2))}. \end{aligned}$$

- ▶ This is exactly the Bradley-Terry form.
- ▶ So BT is not different from softmax mathematically: it is simply the softmax over *two* items.
- ▶ In preference modeling, BT emphasizes the idea of a pairwise comparison and the assumption that comparisons are driven by underlying latent utilities.

Training the Reward Model

- ▶ Since $r^*(x, y)$ is unknown, we train an approximate reward model $r_\phi(x, y)$ using human preference data.
- ▶ The dataset consists of labeled comparisons:

$$\mathcal{D} = \{(x^{(i)}, y_w^{(i)}, y_l^{(i)})\}_{i=1}^N,$$

where $y_w^{(i)}$ is the preferred response and $y_l^{(i)}$ is the less preferred one.

- ▶ Because humans provide relative preferences rather than absolute scores, the reward model is trained from pairwise comparison data.
- ▶ The goal is to learn a scoring function that assigns higher reward to responses that humans prefer.

Pairwise Preference Modeling (again)

- ▶ Human annotations are based on comparisons, not absolute ratings.
- ▶ Instead of predicting a single reward score, the model compares two responses and predicts which one is better.
- ▶ Given two responses y_w (preferred) and y_l (less preferred), we model:

$$P(y_w \succ y_l \mid x) = \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))$$

- ▶ The logistic function $\sigma(\cdot)$ converts the reward difference into a probability between 0 and 1.
- ▶ This gives a pairwise probabilistic objective, similar to binary classification.

The Negative Log-Likelihood (NLL) Loss Function

- ▶ To train the reward model, we use the negative log-likelihood loss:

$$\mathcal{L}_R(r_\phi, \mathcal{D}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} [\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))].$$

- ▶ Breaking down the components:
 - ▶ $r_\phi(x, y) \rightarrow$ The reward model scores a response in a way that helps predict human preferences.
 - ▶ $\sigma(\cdot) \rightarrow$ The logistic function, converting reward differences into ranking probabilities.
 - ▶ The model is trained to assign higher rewards to preferred responses.
- ▶ This loss encourages the model to assign higher probability to the human-preferred ranking.

What Are We Training the Reward Model to Do?

- ▶ For each prompt x , humans compare two responses: y_w (winner) and y_l (loser).
- ▶ Our reward model gives each response a score:

$$r_\phi(x, y_w), \quad r_\phi(x, y_l).$$

- ▶ We want these scores to reflect the human judgment: the preferred response should be more likely to win.
- ▶ So we convert the score difference into a probability:

$$P_\phi(y_w \succ y_l \mid x) = \sigma(r_\phi(x, y_w) - r_\phi(x, y_l)).$$

- ▶ If the preferred response gets a much higher score, this probability is close to 1.
- ▶ So training means: adjust ϕ so that the model assigns high probability to the choices humans actually made.

Maximum Likelihood

- ▶ In each comparison, the model assigns a probability to the human's observed choice.
- ▶ A good model should assign a **high probability** to the preferences that humans actually gave.
- ▶ So we try to choose parameters ϕ that make the observed dataset as probable as possible.
- ▶ This general idea is called **maximum likelihood**: choose the model parameters so that the human preference data looks as unsurprising as possible under the model
- ▶ The negative log-likelihood loss is just the optimization form of this idea.

Why Negative Log-Likelihood?

- ▶ Maximum likelihood means maximizing the probability that the preferred response beats the rejected response, across the observed comparisons:

$$\prod_{(x, y_w, y_l) \in \mathcal{D}} P_{\phi}(y_w \succ y_l \mid x).$$

- ▶ Taking the logarithm turns this product into a sum:

$$\sum_{(x, y_w, y_l) \in \mathcal{D}} \log P_{\phi}(y_w \succ y_l \mid x),$$

- ▶ We add a minus so we minimize a loss.
- ▶ So maximizing likelihood is equivalent to minimizing the negative log-likelihood (NLL):

$$\mathcal{L}_R = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} [\log \sigma(r_{\phi}(x, y_w) - r_{\phi}(x, y_l))] .$$

- ▶ Intuitively: the loss is small when the model gives high probability to the human-preferred response, and large when it does not.

Ranking Two Responses for a Given Prompt

- ▶ Suppose a language model generates two responses for a given prompt x "Do you like pancakes?":
 - ▶ y_w ("I love pancakes.") $\rightarrow$ Preferred by humans.
 - ▶ y_l ("Pancakes are food.") $\rightarrow$ Less preferred.
- ▶ The reward model assigns scores:

$$r_\phi(x, y_w) = 2.0, \quad r_\phi(x, y_l) = 1.2.$$

- ▶ Compute the probability that y_w is preferred:

$$\begin{aligned} P(y_w \succ y_l \mid x) &= \sigma(r_\phi(x, y_w) - r_\phi(x, y_l)) \\ &= \sigma(2.0 - 1.2) = \sigma(0.8) = \frac{1}{1 + e^{-0.8}} \approx 0.69. \end{aligned}$$

- ▶ The negative log-likelihood (NLL) loss for this example:

$$\mathcal{L}_R = -\log(0.69) \approx 0.37.$$

- ▶ Since the model correctly ranks y_w higher, the loss is low.

Incorrect Ranking by the Reward Model

- ▶ Now, suppose the model assigns incorrect reward scores:

$$r_{\phi}(x, y_w) = 1.0, \quad r_{\phi}(x, y_l) = 1.5.$$

- ▶ Compute the probability that y_w is preferred:

$$\begin{aligned} P(y_w \succ y_l \mid x) &= \sigma(r_{\phi}(x, y_w) - r_{\phi}(x, y_l)) \\ &= \sigma(1.0 - 1.5) = \sigma(-0.5) = \frac{1}{1 + e^{0.5}} \approx 0.38. \end{aligned}$$

- ▶ The negative log-likelihood (NLL) loss for this case:

$$\mathcal{L}_R = -\log(0.38) \approx 0.97.$$

- ▶ Since the model ranks y_l higher than y_w , contradicting human labels, the loss is high.

NLL Loss

The model learns by minimizing the loss, adjusting reward scores to match human preferences.

- ▶ Lower loss occurs when the model correctly ranks preferred responses higher.
- ▶ Higher loss occurs when the model assigns incorrect rankings.

If $P(y_w \succ y_l)$ is already high, the log function assigns a low penalty, encouraging stability. If $P(y_w \succ y_l)$ is too low, the log function assigns a high penalty, forcing corrections.

Why Not Use Regression?

- ▶ One could imagine training the reward model to predict an absolute reward score.
- ▶ For example, if true targets $r^*(x, y)$ were available, we could use mean squared error:

$$\mathcal{L}_{\text{MSE}} = \mathbb{E}[(r_{\phi}(x, y) - r^*(x, y))^2].$$

- ▶ But this is usually a poor fit for RLHF:
 - ▶ Human feedback is mainly relative rather than absolute.
 - ▶ People are better at comparing two responses than assigning reliable numeric scores.
 - ▶ Numeric ratings can be inconsistent across annotators.
- ▶ So pairwise comparisons are usually the cleaner supervision signal.

Recap

- ▶ The reward model is trained to rank responses, not to predict absolute reward values.
- ▶ Training uses human preference comparisons between pairs of responses.
- ▶ The loss encourages the model to assign higher rewards to preferred responses than to rejected ones.
- ▶ This avoids relying on inconsistent absolute scoring and gives a more stable training signal.

Reward Scale and Normalization

- ▶ The reward model needs to preserve relative preferences, not absolute reward values.
- ▶ For stability, implementations often center or normalize reward outputs.
- ▶ For example, one may recenter rewards so that their mean on a reference dataset is approximately zero:

$$\mathbb{E}_{x,y \sim \mathcal{D}}[r_{\phi}(x, y)] \approx 0.$$

- ▶ This can help keep optimization stable and reduce reward-scale drift.
- ▶ The exact normalization method varies across implementations.

From Reward Model to RL Fine-Tuning

- ▶ We now have:
 - ▶ A language model (LM) that generates responses.
 - ▶ A reward model (RM) that assigns scalar scores learned from human preference data.
- ▶ Next step: Reinforcement Learning to fine-tune the LM by:
 - ▶ Generating new responses.
 - ▶ Assigning rewards using the reward model.
 - ▶ Updating a policy to maximize expected reward while maintaining stability.

What is a Policy in Reinforcement Learning?

- ▶ A policy $\pi(a|s)$ is a function that defines how an agent chooses actions based on the current state.
- ▶ Formally, it is a probability distribution over actions:
 - ▶ s is the state (context, previous tokens).
 - ▶ a is the action (next token to generate).
 - ▶ $\pi(a|s)$ is the probability of choosing action a given state s .
- ▶ Policies can be:
 - ▶ Deterministic: Always picks the same action for a given state.
 - ▶ Stochastic: Outputs a probability distribution over actions.

How Policies Work in Language Models

- ▶ In RLHF, the policy is the language model (LLM).
- ▶ Given a prompt x , the policy selects a sequence of tokens:

$$\pi_{\theta}(y_t|x, y_1, \dots, y_{t-1})$$

- ▶ Each action y_t is a word or token chosen at step t .
- ▶ The policy is updated using reinforcement learning to improve text generation.

Mapping RL Concepts to Language Models

- ▶ In standard RL:
 - ▶ The state is the agent's observation of the world.
 - ▶ The action is the decision the agent makes.
 - ▶ The reward is the feedback signal guiding learning.
- ▶ In RLHF:
 - ▶ State: The current prompt and past tokens.
 - ▶ Action: The next token the model generates.
 - ▶ Reward: Based on the human-trained reward model.

KL-Regularized Proximal Policy Optimization (PPO) in RLHF

- ▶ Recall: a separate reward model assigns scores to generated text.
- ▶ The reward function is based on the reward model:

$$r(x, y) = r_{\phi}(x, y) - \beta D_{\text{KL}} \left[\pi_{\theta}(y|x) \parallel \pi^{\text{SFT}}(y|x) \right]$$

where:

- ▶ π_{θ} is the current policy that is continuously updated during RLHF training to maximize the reward function.
- ▶ π^{SFT} is the supervised fine-tuned model, used as a reference.
- ▶ At the start of RLHF: $\pi_{\theta}^{(0)} = \pi^{\text{SFT}}$, but π_{θ} evolves as training progresses.
- ▶ $r_{\phi}(x, y)$ is the human-trained reward model that provides feedback on text quality.
- ▶ $D_{\text{KL}} [\pi_{\theta} \parallel \pi^{\text{SFT}}]$ discourages the policy from drifting too far from the reference SFT model.

KL-Regularized RLHF: Simple Analogy

Imagine revising an essay draft:

- ▶ Reward: *make it better*
- ▶ KL penalty: *do not rewrite it so much that it stops sounding like the original assignment*

Main idea

Improve the output, but keep it recognizable and under control.

This is the role of the KL term in RLHF: it prevents the policy from changing too aggressively while still encouraging higher-reward responses.

Per-Response Reward in RLHF

- ▶ For a prompt x , the current policy π_θ generates a response y .
- ▶ The reward model assigns that response a score $r_\phi(x, y)$.
- ▶ To discourage the policy from drifting too far from the reference model π^{SFT} , we add a KL-based penalty.
- ▶ So the reward for one sampled response is:

$$r(x, y) = r_\phi(x, y) - \beta \text{KL penalty.}$$

- ▶ Intuition:
 - ▶ high reward-model score $\rightarrow$ encourage this behavior
 - ▶ large deviation from the reference model $\rightarrow$ penalize it

From a Single Response to a Policy Objective

- ▶ For one prompt x and one generated response y , the reward model gives a score $r_\phi(x, y)$.
- ▶ But the policy π_θ does not produce just one fixed response: it defines a distribution over many possible responses.
- ▶ So we should not judge the policy by one sample alone.
- ▶ Instead, we evaluate it by the average reward of the responses it generates.
- ▶ This leads to the objective

$$\max_{\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(y|x)} [r_\phi(x, y)].$$

- ▶ The expectation appears because we care about how the policy performs on average over prompts and sampled outputs.

The Role of β in RLHF

- ▶ The RLHF objective function balances:

$$\mathbb{E}[r_\phi(x, y)] - \beta D_{\text{KL}} \left[\pi_\theta(y|x) \parallel \pi^{\text{SFT}}(y|x) \right]$$

- ▶ Two competing terms:
 - ▶ $\mathbb{E}[r_\phi(x, y)]$: Maximizing alignment with human preferences.
 - ▶ $-\beta D_{\text{KL}} [\pi_\theta \parallel \pi^{\text{SFT}}]$: Penalizing divergence from the initial model to retain generalization.
- ▶ What does β control?
 - ▶ High $\beta \rightarrow$ Strong KL penalty, restricting deviation from the supervised fine-tuned model.
 - ▶ Low $\beta \rightarrow$ The policy can deviate more freely, which increases the risk of reward hacking or undesirable drift.

Kullback–Leibler (KL) Divergence

- ▶ KL divergence measures how one probability distribution differs from another.
- ▶ For two discrete distributions P and Q :

$$D_{\text{KL}}(P\|Q) = \sum_x P(x) \log \left(\frac{P(x)}{Q(x)} \right)$$

- ▶ Properties:
 - ▶ $D_{\text{KL}}(P\|Q) \geq 0$
 - ▶ $D_{\text{KL}}(P\|Q) = 0$ if and only if $P = Q$
 - ▶ Not symmetric: $D_{\text{KL}}(P\|Q) \neq D_{\text{KL}}(Q\|P)$
- ▶ Commonly used in machine learning to measure how much a model distribution deviates from a target.

KL Divergence Example: Travel Times

- ▶ Suppose we observe these travel times (in hours) to Lagerstrasse: 1, 2, 3 hours
 - ▶ Half of us take 1 hour, 30% of us take 2, and 20% of us take 3
 - ▶ Our model predicts instead that 40% of us take 1 hour and 40% of us take 2; it correctly predicts that 20% of us take 3
- ▶ True distribution P : [0.5, 0.3, 0.2]
Model distribution Q : [0.4, 0.4, 0.2]
- ▶ Compute $D_{\text{KL}}(P\|Q)$ using base 2 (bits):

$$\begin{aligned}D_{\text{KL}}(P\|Q) &= 0.5 \log_2 \left(\frac{0.5}{0.4} \right) + 0.3 \log_2 \left(\frac{0.3}{0.4} \right) + 0.2 \log_2 \left(\frac{0.2}{0.2} \right) \\&\approx 0.5(0.3219) + 0.3(-0.4150) + 0 \\&\approx 0.1609 - 0.1245 = \boxed{0.0364 \text{ bits}}\end{aligned}$$

- ▶ Interpretation: Small divergence $\rightarrow$ model is close to true distribution.

KL Divergence Example: Travel Times

- Suppose the true distribution of travel times to Lagerstrasse is

$$P = [0.5, 0.3, 0.2],$$

while our model predicts

$$Q = [0.85, 0.10, 0.05].$$

- Then

$$D_{\text{KL}}(P\|Q) = \sum_i P(i) \log_2 \frac{P(i)}{Q(i)}.$$

- Plugging in the values:

$$\begin{aligned} D_{\text{KL}}(P\|Q) &= 0.5 \log_2 \left(\frac{0.5}{0.85} \right) + 0.3 \log_2 \left(\frac{0.3}{0.10} \right) + 0.2 \log_2 \left(\frac{0.2}{0.05} \right) \\ &\approx -0.3828 + 0.4755 + 0.4 \\ &= \boxed{0.4927 \text{ bits}} \end{aligned}$$

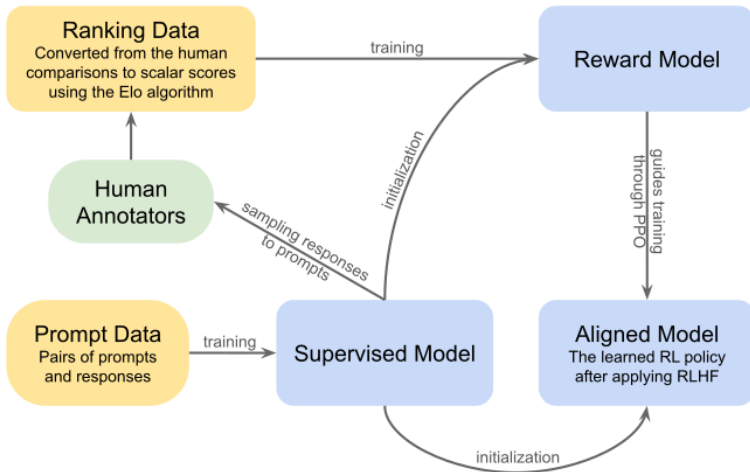
- The bigger divergence confirms that the model is worse now

RLHF Overview

Key Steps:

1. Gather and prepare Prompt Data
 - Pairs of (prompt, response) examples.
2. Supervised Fine-Tuning
 - Train or fine-tune a baseline model on these prompt/response pairs.
3. Collect Human Preference Data
 - Have human annotators compare different model outputs.
4. Train a Reward Model
 - Use human preference comparisons to train a model that assigns higher scores to preferred responses.
5. Optimize with RL (e.g., PPO)
 - Use the Reward Model as the reward signal, fine-tune to get an aligned policy.

RLHF as a Fine-Tuning Process



Source: Wikipedia

RLHF Diagram

Explanation:

- ▶ Prompt Data → Supervised Model: trains an initial policy on known good responses.
- ▶ Human Annotators: produce pairwise rankings of model outputs.
- ▶ Ranking Data: used to train a reward model from pairwise human preferences.
- ▶ Reward Model: predicts how humans would rank a given response.
- ▶ Aligned Model: optimized via RL (e.g., PPO) with the Reward Model guiding the policy updates.

OpenAI's visual summary

30k tasks!

Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

Explain the moon landing to a 6 year old

A labeler demonstrates the desired output behavior.

Some people went to the moon...

This data is used to fine-tune GPT-3 with supervised learning.

SFT

Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.

Explain the moon landing to a 6 year old

A Explain gravity... B Explain why... C Moon is natural satellite of... D People went to the moon...

A labeler ranks the outputs from best to worst.

D > C > A = B

This data is used to train our reward model.

RM

Step 3

Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

Write a story about frogs

The policy generates an output.

PPO

Once upon a time...

The reward model calculates a reward for the output.

RM

The reward is used to update the policy using PPO.

r_k

[Ouyang et al., 2022]

Source: OpenAI